

UNIVERSIDAD COMPLUTENSE DE MADRID
FACULTAD DE CIENCIAS FÍSICAS

DEPARTAMENTO DE ARQUITECTURA DE COMPUTADORES Y AUTOMÁTICA



TRABAJO DE FIN DE GRADO

Código de TFG: ACA02

Navegación coordinada en sistemas multiagente

Coordinated navigation in multiagent systems

Supervisores: Lía García Pérez, Juan Jiménez

Lucas Balfagón Zapatero

Grado en Física

Curso académico 2025-26

Convocatoria extraordinaria

[Navegación coordinada en sistemas multiagente]

Resumen:

En este trabajo se estudia e implementa un algoritmo de navegación coordinada para sistemas multiagente basado en campos vectoriales de guiado. El campo se construye sobre un espacio ampliado con una coordenada virtual que parametriza la curva objetivo, lo que evita la aparición de puntos singulares y permite coordinar la posición de los robots sobre la trayectoria. El campo de seguimiento se deduce mediante álgebra lineal, evitando el producto exterior empleado en la literatura y el término dependiente de la dimensión que este introduce. La convergencia del sistema a la curva y a las distancias deseadas entre agentes se demuestra analíticamente mediante el método de estabilidad de Lyapunov y el principio de invariancia de LaSalle. Para evitar las colisiones entre agentes, se incorpora un algoritmo de evitación basado en funciones de barrera de control, que altera mínimamente el campo resolviendo un problema de optimización cuadrática. Los resultados confirman la convergencia del guiado coordinado, mientras que la introducción de las barreras incumple los objetivos de control originales, pero da lugar a trayectorias aparentemente periódicas y estables, cuya desviación disminuye al relajar la condición de seguridad.

Abstract:

This work studies and implements a coordinated navigation algorithm for multiagent systems based on Guiding Vector Fields (GVF). The field is built on an augmented space with a virtual coordinate that parameterizes the target path, which prevents the appearance of singular points and allows the robots' positions along the path to be coordinated. The path-following field is derived through linear algebra, avoiding the wedge product used in the literature and the dimension-dependent term it introduces. The convergence of the system to the curve and to the desired inter-agent distances is proved analytically by means of Lyapunov's stability method and LaSalle's invariance principle. To prevent collisions between agents, an avoidance algorithm based on control barrier functions is incorporated, which modifies the field minimally by solving a quadratic optimization problem. The results confirm the convergence of the coordinated guidance, whereas the introduction of the barriers fails to meet the original control objectives but gives rise to apparently periodic and stable trajectories, whose deviation decreases as the safety condition is relaxed.

Declaración Responsable sobre Autoría y Uso Ético de Herramientas de Inteligencia Artificial (IA)

Yo, **Lucas Balfagón Zapatero**,

con DNI **47294613N**,

declaro de manera responsable que el/la presente:

☒ **Trabajo de Fin de Grado (TFG)**

Trabajo de Fin de Máster (TFM)

☐ Tesis Doctoral

Titulado:

Navegación coordinada en sistemas multiagente,

es el resultado de mi trabajo intelectual personal y creativo, y ha sido elaborado de acuerdo con los principios éticos y las normas de integridad vigentes en la comunidad académica y, más específicamente, en la Universidad Complutense de Madrid.

Soy, pues, autor del material aquí incluido y, cuando no ha sido así y he tomado el material de otra fuente, lo he citado o bien he declarado su procedencia de forma clara -incluidas, en su caso, herramientas de inteligencia artificial-. Las ideas y aportaciones principales incluidas en este trabajo, y que acreditan la adquisición de competencias, son mías y no proceden de otras fuentes o han sido reescritas usando material de otras fuentes.

Asimismo, aseguro que los datos y recursos utilizados son legítimos, verificables y han sido obtenidos de fuentes confiables y autorizadas. Además, he tomado medidas para garantizar la confidencialidad y privacidad de los datos utilizados, evitando cualquier tipo de sesgo o discriminación injusta en el tratamiento de la información.

En **Madrid**, a **26 de mayo de 2026**.

Firma



1. Introducción

El control sobre la trayectoria de robots es una herramienta muy útil para resolver ciertos tipos de labores, como la monitorización ambiental, la vigilancia del tráfico en carretera, la inspección de infraestructuras o las tareas de búsqueda y rescate [1]. Se han propuesto numerosos algoritmos para resolver este problema, pero este trabajo se centra particularmente en los campos de guiado (en inglés, Guiding Vector Fields, GVF). Estos tratan la trayectoria objetivo como un objeto geométrico, sin dependencia temporal. Esto supone una ventaja frente a otros algoritmos, como el seguimiento de un punto de referencia que avanza sobre la curva en el tiempo [2]. Al no estar el robot obligado a alcanzar una posición concreta en cada instante, puede recorrer la curva con buena precisión y poco esfuerzo de control [3].

Para el algoritmo de estudio, se define una coordenada virtual ω que parametriza la curva a la que deben converger los robots, y que se incorpora como una dimensión adicional del estado del sistema. Esto evita la aparición de puntos singulares en los campos de guiado [4], además de permitir la coordinación entre agentes, como podrá comprobarse más adelante.

Para evitar las colisiones entre robots a la hora de converger a la curva y recorrerla, también deben implementarse algoritmos de evitación de obstáculos. Algunos ejemplos de este tipo de algoritmos son los campos de potencial artificial [5], u otros enfoques especialmente diseñados para campos de guiado, como el de [6], que combina el campo de seguimiento con un campo reactivo mediante funciones de mezcla suaves. Para este trabajo, se recurre a las funciones de barrera de control [7]. Estas alteran mínimamente el campo de cada agente para evitar las colisiones, recurriendo a multiplicadores de Lagrange que calculan la variación más pequeña necesaria para cumplir la condición de seguridad elegida.

2. Objetivos

El primer objetivo será implementar y estudiar el algoritmo de guiado coordinado mediante campos vectoriales de guiado, propuesto en [8]. También se desarrollará la demostración analítica de su convergencia, obtenida de [9], recurriendo al método de estabilidad de Lyapunov y al principio de invariancia de LaSalle. Además, se empleará el algoritmo de evitación de obstáculos mediante funciones de barrera de control [7], estudiando su efecto sobre los campos de guiado y atendiendo a las posibles dificultades que pueda introducir en la convergencia a las trayectorias y posiciones deseadas.

3. Campos vectoriales de guiado

En este primer apartado se contruye el campo vectorial que hará que los agentes converjan y sigan a la trayectoria objetivo, definido en [8]. Esta trayectoria objetivo será una curva parametrizada.

En primer lugar, debe definirse el espacio de trayectorias donde se desarrolla la dinámica del sistema. Este tiene n coordenadas “reales” x_i , $i = \{1, \dots, n\}$, que se corresponden con el espacio físico donde los agentes se desplazarán en un escenario real. También consta de una coordenada virtual extra, que denotamos como ω . Esta coordenada virtual, además de ser una dimensión del espacio de trayectorias, es también el parámetro de la curva objetivo $\mathbf{f}(\omega) : \mathbb{R} \rightarrow \mathbb{R}^n$, que se representa sobre las dimensiones espaciales. Las posiciones en este espacio se definen como $\boldsymbol{\xi} = (x_1, \dots, x_n, \omega) \in \mathbb{R}^{n+1}$. Como aclaración conceptual, al referirse a coordenadas reales, se hace referencia al subespacio de n dimensiones que contiene a la curva objetivo $\mathbf{f}(\omega)$, no a que pertenezcan a $\mathbb{R}$.

Se define también el error entre la posición del robot y la posición deseada, $\phi_i : \mathbb{R}^{n+1} \rightarrow \mathbb{R}$, $\phi_i(\boldsymbol{\xi}) = x_i - f_i(\omega)$. Dado que puede definirse la trayectoria sobre la curva como $x_i = f_i(\omega)$, $\forall i$, la intersección de las superficies de nivel 0 de $\phi_i(\boldsymbol{\xi})$ definen la trayectoria objetivo en el espacio de $n+1$ dimensiones:

$$C := \{\boldsymbol{\xi} \in \mathbb{R}^{n+1} : \phi_i(\boldsymbol{\xi}) = 0, i \in \{1, \dots, n\}\} \quad (1)$$

La proyección de C sobre las coordenadas reales es la curva parametrizada $\mathbf{f}(\omega)$. En la figura 1 se muestra esto para la curva $\mathbf{f}(\omega) = [\cos(\omega), \sin(\omega)]$:

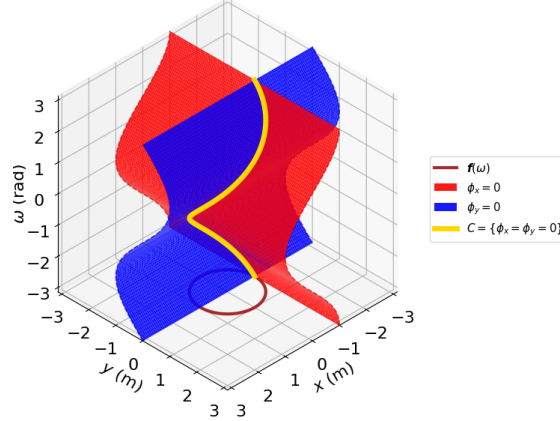


Figura 1: Visualización de la intersección de las superficies $\phi_x = \phi_y = 0$, junto con su proyección sobre el plano de coordenadas físicas, para la curva $\mathbf{f}(\omega) = [\cos(\omega), \sin(\omega)]$.

Este mismo diagrama no podría representarse para el caso de $n = 3$, ya que el espacio de trayectorias pasaría a tener cuatro dimensiones. Sin embargo, la aplicación es análoga al caso bidimensional. Todas las simulaciones de este trabajo son para trayectorias $\mathbf{f}(\omega)$ bidimensionales, con $n = 2$. Aún así, los algoritmos implementados en el trabajo son generalizables a curvas objetivo tridimensionales.

Una vez definido el espacio donde se desarrolla la dinámica y las funciones ϕ_i , puede construirse el campo de guiado. Este debe cumplir dos objetivos de control. Las trayectorias deben tender a la curva objetivo $\mathbf{f}(\omega)$, y una vez en la curva, el campo vectorial debe ser tangente a la misma. De esta forma, el robot no solo llega a la curva, sino que se desplaza sobre la misma sin salirse. El campo vectorial constará de dos términos, cada uno asociado a cada objetivo. Un último apunte sobre notación; para los vectores que definen el campo, se emplea la flecha $\vec{v}$, mientras que el resto de vectores se denotan en negrita $\mathbf{v}$.

3.1. Guiado hacia la trayectoria deseada

El primer término se encargará de la convergencia a las curvas de nivel $\phi_i = 0$, denotadas como $\vec{v}^{conv}(\boldsymbol{\xi})$. Se toma el producto de $-\nabla\phi_i$ por ϕ_i . De esta forma, para curvas de nivel que cumplan $\phi_i > 0$, el campo vectorial apuntará en dirección descendente, y lo hará al contrario para curvas de nivel que cumplan $\phi_i < 0$. Al interpretar estas funciones como superficies definidas únicamente por los parámetros de los que dependen, x_i y ω_i , se pasa de tener una hipersuperficie de $n+1$ dimensiones, a una superficie representable gráficamente, $\phi_i(\boldsymbol{\xi}) \rightarrow \phi_i(x_i, \omega)$. Así podemos observar el comportamiento de cada componente real respecto a la coordenada virtual ω :

$$-\phi_i(x_i, \omega) \cdot \nabla\phi_i(x_i, \omega) = -\phi_i \cdot \left[\frac{d}{dx_i}(x_i - f_i(\omega)), \frac{d}{d\omega}(x_i - f_i(\omega)) \right] = [-\phi_i, \phi_i f'_i(\omega)] \quad (2)$$

En la figura 2 se muestra como ejemplo el campo vectorial definido por (2) sobre las superficies $\phi_x(x, \omega)$ y $\phi_y(y, \omega)$, junto con la curva de nivel $\phi_i(x_i, \omega) = 0, \forall i$, con la curva parametrizada $\mathbf{f}(\omega) = [\cos(\omega), \sin(\omega)]$:

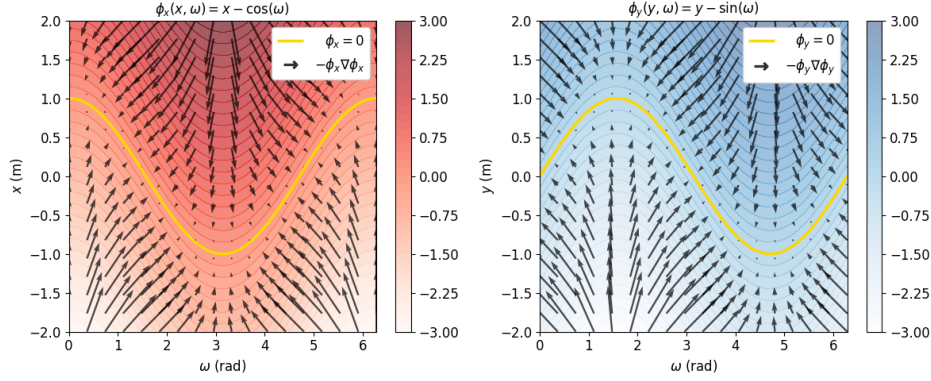


Figura 2: Representación de las superficies $\phi_x = x - f_x(\omega)$ (izquierda) y $\phi_y = y - f_y(\omega)$ (derecha), para $\mathbf{f}(\omega) = [\cos(\omega), \sin(\omega)]$. Sobre la superficie, se dibujan las curvas de nivel $\phi = 0$, junto con el campo vectorial $-\phi_i \nabla \phi_i$.

En la figura 2 puede observarse cómo es el comportamiento del gradiente para cada componente del campo de convergencia. La dinámica lleva a cada condición inicial al punto más cercano a la curva objetivo. Ahora se aplica el gradiente al campo total, dependiente de las $n+1$ variables:

$$-\phi_i(\boldsymbol{\xi}) \nabla \phi_i(\boldsymbol{\xi}) = \left[\begin{array}{c} 0 \\ \vdots \\ -\phi_i(\boldsymbol{\xi}) \\ \vdots \\ 0 \\ \phi_i(\boldsymbol{\xi}) f'_i(\omega) \end{array} \right] \left. \vphantom{\begin{array}{c} 0 \\ \vdots \\ -\phi_i(\boldsymbol{\xi}) \\ \vdots \\ 0 \\ \phi_i(\boldsymbol{\xi}) f'_i(\omega) \end{array}} \right\} i \text{ elementos} \quad (3)$$

Al definir el campo vectorial para las n componentes espaciales, se debe sumar la aportación de cada coordenada espacial. Así, el campo de convergencia $\vec{v}^{conv} : \mathbb{R}^{n+1} \rightarrow \mathbb{R}^{n+1}$, queda definido como:

$$\vec{v}^{conv}(\boldsymbol{\xi}) = - \sum_{i=1}^n k_i \phi_i(\boldsymbol{\xi}) \nabla \phi_i(\boldsymbol{\xi}) = \left[\begin{array}{c} -k_1 \phi_1(\boldsymbol{\xi}) \\ \vdots \\ -k_n \phi_n(\boldsymbol{\xi}) \\ \sum_{i=1}^n k_i \phi_i(\boldsymbol{\xi}) f'_i(\omega) \end{array} \right] \quad (4)$$

Donde se han añadido unas ganancias k_i que definen la velocidad de convergencia, cuyo efecto se discute más adelante. Así, queda definido el primer término del campo vectorial. Este puede interpretarse como una respuesta proporcional al error entre las posiciones x_i y la curva $\mathbf{f}(\omega)$. Si todos las coordenadas x_i se encuentran sobre la curva, los elementos de este campo se anulan y deja de haber una aportación al campo total. Sin embargo, si no se encuentran sobre la curva, el campo define una trayectoria para llevarlos a la misma. En resumen, la velocidad de aproximación será más lenta al acercarnos a la trayectoria deseada, como se ve en la figura 2.

3.2. Seguimiento de la curva

En esta sección se define la segunda aportación al campo total, que se denota $\vec{v}^{seg}(\xi)$. Debe encargarse de que los robots sigan la trayectoria una vez llegan a la misma. Por ello, el campo vectorial de guiado sobre la curva $\mathbf{f}(\omega)$ debe ser tangente a la misma.

Para cumplir estos requisitos, se recurre al álgebra lineal. Para encontrar un campo tangente a la curva, $\vec{v}^{seg} : \mathbb{R}^{n+1} \rightarrow \mathbb{R}^{n+1}$ debe ser el complemento ortogonal de los gradientes, $\vec{v}^{seg} \perp \text{lin}\{\nabla\phi_1, \dots, \nabla\phi_n\}$. Para ello, debe resolverse el siguiente sistema lineal de ecuaciones:

$$\begin{bmatrix} \nabla\phi_1(\xi) \\ \vdots \\ \nabla\phi_n(\xi) \end{bmatrix} \vec{v}^{seg}(\xi) = \begin{bmatrix} 1 & 0 & \dots & 0 & -f'_1(\omega) \\ 0 & 1 & \dots & 0 & -f'_2(\omega) \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & -f'_n(\omega) \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_n \\ v_{n+1} \end{bmatrix} = \vec{0} \quad (5)$$

Al tener n ecuaciones independientes, y $n + 1$ incógnitas, el espacio de soluciones tendrá dimension 1. El sistema tiene como soluciones:

$$\vec{v}^{seg}(\xi) = \lambda \begin{bmatrix} f'_1(\omega) \\ \vdots \\ f'_n(\omega) \\ 1 \end{bmatrix}, \quad \lambda \in \mathbb{R} \quad (6)$$

El vector tiene como elementos las componentes tangentes a la curva $\mathbf{f}(\omega)$, salvo la última componente, que representa el ritmo de avance en ω . De esta forma, el parámetro λ caracteriza la velocidad con la que se recorre la curva, $\dot{\omega}$. Para $\lambda < 0$, los vectores tangentes cambian su sentido, y $\dot{\omega} < 0$, recorriéndose la curva en sentido contrario.

Esta deducción del campo supone una mejora sobre el enfoque del algoritmo en la literatura [8]. En la fuente bibliográfica original, para obtener un campo perpendicular a $\nabla\phi_i$, se recurre al producto exterior (en inglés, wedge product). La operación es una generalización del producto vectorial para cualquier número de dimensiones del espacio, permitiendo calcular el vector perpendicular a una familia de vectores arbitraria. Sin embargo, al recurrir a un determinante para resolverlo, aparece un término $(-1)^n$ en las ecuaciones de la dinámica, donde n representa las dimensiones reales, como se comentó previamente. Esto se debe a que el número de menores del determinante depende de n , por lo que cambia el signo al cambiar el número de dimensiones. Sin embargo, como se ha podido demostrar, este término puede evitarse, ahorrando así potencia computacional y definiendo el término λ para escoger la velocidad sobre la curva.

3.3. Implementación de ambos campos vectoriales

Ahora se construye el campo de guiado con los campos de convergencia y seguimiento de la curva, $\vec{v}^{guia} : \mathbb{R}^{n+1} \rightarrow \mathbb{R}^{n+1}$.

$$\vec{v}^{guia} = \vec{v}^{conv} + \vec{v}^{seg} = \begin{bmatrix} -k_1\phi_1(\xi) + \lambda f'_1(\omega) \\ \vdots \\ -k_n\phi_n(\xi) + \lambda f'_n(\omega) \\ \sum_{i=1}^n (k_i\phi_i(\xi)f'_i(\omega)) + \lambda \end{bmatrix} \quad (7)$$

Para representar las dinámicas de los agentes, debe resolverse numéricamente el siguiente sistema de $n + 1$ ecuaciones diferenciales ordinarias:

$$\dot{\xi} = \vec{v}^{guia}(\xi) \quad (8)$$

De estas ecuaciones se obtienen las curvas integrales correspondientes a cada condición inicial. Sin embargo, si quisiera simularse la dinámica de un vehículo real, esta solución no sería suficiente. Se deben calcular las ecuaciones del movimiento del vehículo, e introducir también sus tolerancias. Un vehículo real tiene orientación, ciertos ángulos de giro permitidos, aceleraciones y velocidades máximas, etc. Para simularlos, el campo definido se emplea como guía para el vehículo, que tiene implementado sistemas de control que le hacen converger a las curvas integrales del campo. Este aspecto queda fuera del foco de estudio del trabajo, pero es importante remarcar que lo que se está resolviendo es el caso de un vehículo ideal sin restricciones.

3.4. Simulación del campo de guiado

Ahora se simulan las trayectorias para diferentes ganancias k_x y k_y , estudiando así su efecto en las trayectorias. Las trayectorias representadas son la proyección sobre el espacio de coordenadas real. Para entender mejor cómo evoluciona la dinámica del sistema, se recomienda acceder al repositorio de este TFG [10], donde pueden encontrarse tanto las simulaciones, como las animaciones de las trayectorias. Se toma una curva paramétrica arbitraria, $f(\omega) = [\frac{\omega}{2} \cos(2,4\omega), \frac{\omega}{1,5} \sin(\omega)]$. Las condiciones iniciales de cada trayectoria son aleatorias. Estas son las trayectorias obtenidas:

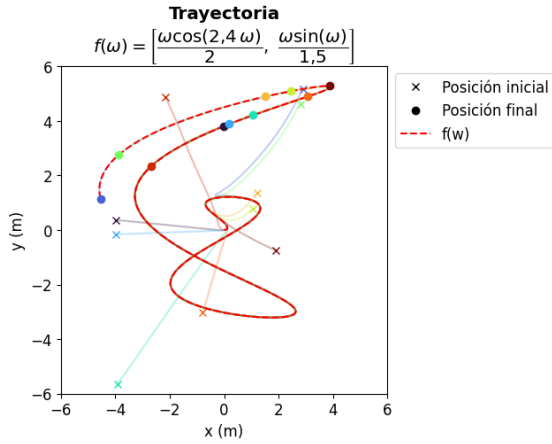


Figura 3: Trayectorias con 8s de tiempo de integración, para 10 condiciones iniciales aleatorias. Con ganancias $k_x = 5s^{-1}$, $k_y = 5s^{-1}$ y $\lambda = 1rad s^{-1}$

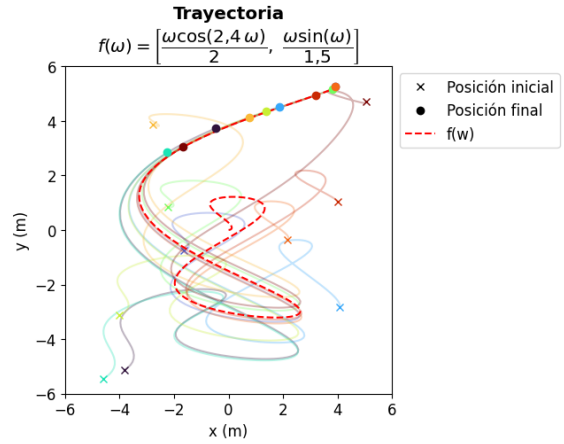


Figura 4: Trayectorias con 8s de tiempo de integración, para 5 condiciones iniciales aleatorias. Con ganancias $k_x = 0,1s^{-1}$, $k_y = 0,1s^{-1}$ y $\lambda = 1rad s^{-1}$

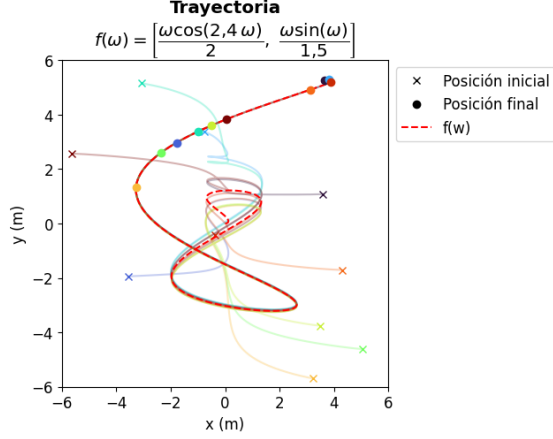


Figura 5: Trayectorias con 8s de tiempo de integración, para 10 condiciones iniciales aleatorias. Con ganancias $k_x = 5s^{-1}$, $k_y = 0,5s^{-1}$ y $\lambda = 1rad s^{-1}$

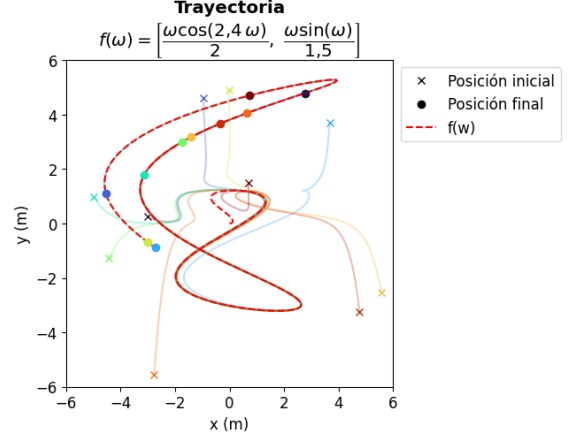


Figura 6: Trayectorias con 8s de tiempo de integración, para 10 condiciones iniciales aleatorias. Con ganancias $k_x = 0,5s^{-1}$, $k_y = 5s^{-1}$ y $\lambda = 1rad s^{-1}$

Pese a que el tiempo simulado sea el mismo, la gráfica solo dibuja la curva $f(\omega)$ hasta la coordenada ω más grande que haya alcanzado alguna de las trayectorias. La longitud que recorran sobre la curva dependerá del punto de la misma sobre el que converjan y el tiempo que tardan en hacerlo, ya que la velocidad de desplazamiento sobre la misma es constante para todas las trayectorias.

Se ve que a ganancias mayores, la convergencia es más rápida. Al crear un desequilibrio entre las ganancias, se observa como la coordenada con la ganancia mayor logra alcanzar mucho antes su posición correspondiente en la curva parametrizada. También se observa que las trayectorias con ganancias bajas, a la par que convergen a la curva, dibujan una trayectoria similar a la misma. Esto se debe a que las ganancias representan la proporción entre el campo de convergencia y el de guiado. Para ganancias pequeñas, el efecto del campo tangencial es notorio incluso en puntos alejados de la curva.

4. Componente de control sobre las posiciones en la curva

En esta sección se implementa un nuevo campo vectorial, que se encargará de coordinar a los agentes, logrando que estos mantengan una distancia determinada entre ellos sobre la curva, medida con el propio parámetro ω .

Se denotan las coordenadas virtuales de cada robot como $\omega_r \in \mathbb{R}$, $r = \{1, \dots, N_R\}$; donde el subíndice r hace referencia a cada uno de los N_R robots que formarán parte del sistema. En el vector $\boldsymbol{\omega} = [\omega_1, \dots, \omega_{N_R}] \in \mathbb{R}^{N_R}$ se dispone la coordenada virtual de cada agente. Para codificar las distancias deseadas entre agentes, se representan en el vector $\boldsymbol{\omega}^* = [\omega_1^*, \dots, \omega_{N_R}^*] \in \mathbb{R}^{N_R}$, donde las coordenadas $\omega_r^* \in \mathbb{R}$ toman al primer agente de la cadena como sistema de referencia:

$$\boldsymbol{\omega}^* = [(\omega_1 - \omega_1)^*, (\omega_2 - \omega_1)^*, \dots, (\omega_{N_R} - \omega_1)^*]^T = [0, \omega_2^*, \dots, \omega_{N_R}^*]^T \quad (9)$$

Una vez definidos estos conceptos, puede introducirse el campo encargado de la coordinación en la curva, $\vec{v}^{coord} : \mathbb{R}^{N_R} \rightarrow \mathbb{R}^{n+1}$:

$$\vec{v}_r^{coord}(\boldsymbol{\omega}) = [0, \dots, 0, c_r(\boldsymbol{\omega})]^T \quad (10)$$

4.1. Definición de la componente $c_r(\omega)$

La función $c_r : \mathbb{R}^{N_R} \rightarrow \mathbb{R}$ es un controlador que se encarga de que el agente r mantenga una distancia determinada respecto a los robots del sistema que se consideren. A este conjunto de robots vecinos respecto de los cuáles debe mantenerse una distancia, se le denomina $\mathcal{N}_r$. El controlador $c_r(\omega)$ se define como:

$$c_r(\omega) = - \sum_{r' \in \mathcal{N}_r} (\omega_r - \omega_{r'} - \Delta_{r,r'}), \quad \forall r \in \{1, \dots, N_R\} \quad (11)$$

$$\Delta_{r,r'} = \omega_r^* - \omega_{r'}^*$$

Para el algoritmo de control de este trabajo, $\mathcal{N}_r$ contiene a dos robots para todo r . El robot r mantendrá su posición respecto de los robots adyacentes en la cadena, que serán los agentes $r+1$ y $r-1$. Los únicos agentes que incumplen esta norma son el primer y último de la cadena. Estos no solo mantendrán la distancia con su único vecino, sino también entre ellos, siendo la distancia objetivo para estos la longitud total de la cadena que forman los agentes. Las componentes de control $c_r(\omega)$ quedan:

$$r \in \{2, \dots, N_R - 1\} : \quad c_r(\omega) = -[(\omega_r - \omega_{r+1} - \Delta_{r,r+1}) + (\omega_r - \omega_{r-1} - \Delta_{r,r-1})]$$

$$r = 1 : \quad c_1(\omega) = -[(\omega_1 - \omega_2 - \Delta_{1,2}) + (\omega_1 - \omega_{N_R} - \Delta_{1,N_R})] \quad (12)$$

$$r = N_R : \quad c_{N_R}(\omega) = -[(\omega_{N_R} - \omega_{N_R-1} - \Delta_{N_R,N_R-1}) + (\omega_{N_R} - \omega_1 - \Delta_{N_R,1})]$$

Estas componentes aumentan o disminuyen la velocidad de forma proporcional al desvío respecto de la distancia deseada. Si el primer y segundo robot están más lejos que Δ_{12} , el robot 1 disminuirá la velocidad, y el segundo la aumentará. Si estuvieran más cerca de lo deseado, darían la respuesta contraria.

Resulta conveniente expresar el conjunto de componentes $c_r(\omega)$ de forma matricial. Se observa que c_r es la diferencia entre las coordenadas de dos robots vecinos, menos la distancia objetivo que deben mantener. Si se define el error respecto a la configuración de referencia como $\tilde{\omega} = \omega - \omega^*$, se puede representar el enlace entre robots con la matriz $D \in \mathbb{R}^{N_R \times N_R}$ [8]. Cuando se habla de enlace entre agentes, significa que hay comunicación entre esos dos agentes. Esto quiere decir que intercambian la información de sus posiciones ω_r . Al conocerlas, pueden desplazarse respecto de estas para converger a la distancia deseada entre agentes. Cada columna de D corresponde a un enlace entre dos robots vecinos, y contiene un $+1$ y un -1 en las posiciones de los robots que une. De este modo, el producto $D^T \tilde{\omega}$ genera el vector de distancias entre vecinos $\Delta \in \mathbb{R}^{N_R}$, que tendrá tantos elementos como enlaces entre robots:

$$\Delta^* = D^T \omega^* = \begin{bmatrix} 1 & -1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & -1 & \cdots & 0 & 0 \\ 0 & 0 & 1 & \cdots & 0 & 0 \\ \vdots & & & \ddots & & \vdots \\ 0 & 0 & 0 & \cdots & 1 & -1 \\ -1 & 0 & 0 & \cdots & 0 & 1 \end{bmatrix} \begin{bmatrix} \omega_1^* \\ \omega_2^* \\ \omega_3^* \\ \vdots \\ \omega_{N_R-1}^* \\ \omega_{N_R}^* \end{bmatrix} = \begin{bmatrix} \omega_1^* - \omega_2^* \\ \omega_2^* - \omega_3^* \\ \vdots \\ \omega_{N_R}^* - \omega_{N_R-1}^* \\ \omega_{N_R}^* - \omega_1^* \end{bmatrix} \quad (13)$$

$$\Delta^* = [\Delta_{1,2}, \Delta_{2,3}, \dots, \Delta_{N_R-1,N_R}, \Delta_{N_R,1}]^T$$

Con esta notación, el conjunto de componentes de coordinación se escribe de forma compacta como:

$$\mathbf{c}(\boldsymbol{\omega}) = -L(\boldsymbol{\omega} - \boldsymbol{\omega}^*) = -L\tilde{\boldsymbol{\omega}}, \quad L = DD^T \quad (14)$$

Siendo $\mathbf{c} : \mathbb{R}^{N_R} \rightarrow \mathbb{R}^{N_R}$, $\mathbf{c}(\boldsymbol{\omega}) = [c_1(\boldsymbol{\omega}), \dots, c_{N_R}(\boldsymbol{\omega})]$ y $L \in \mathbb{R}^{N_R \times N_R}$. Solo queda implementar esta componente de control de distancias al campo de guiado definido en (7). Así, queda definido el campo $\vec{w}_r : \mathbb{R}^{N_R} \rightarrow \mathbb{R}^{n+1}$:

$$\vec{w}_r(\boldsymbol{\xi}) = \vec{v}_r^{guia}(\boldsymbol{\xi}) + k_c \vec{v}_r^{coor}(\boldsymbol{\xi}) = \begin{bmatrix} -k_1\phi_1(\boldsymbol{\xi}) + \lambda f'_1(\omega) \\ \vdots \\ -k_n\phi_n(\boldsymbol{\xi}) + \lambda f'_n(\omega) \\ \sum_{i=1}^n (k_i\phi_i(\boldsymbol{\xi})f'_i(\omega)) + \lambda + k_c c_r(\boldsymbol{\omega}) \end{bmatrix} \quad (15)$$

Donde vuelve a introducirse una nueva ganancia, k_c , que caracteriza la velocidad con la que los robots convergen a la configuración de distancias deseada. Así, queda definido el campo de guiado coordinado al completo, implementando la convergencia a la curva, el seguimiento de la misma y la distancia entre robots en la curva.

4.2. Simulación del campo de guiado con control de distancias

En la figura 7 se muestra la simulación para 5 robots que deben converger a la curva $\mathbf{f}(\omega) = [\cos(3\omega), \sin(\omega)]$, manteniendo una distancia entre agentes de $\Delta_{r,r'} = 0,5$ rad.

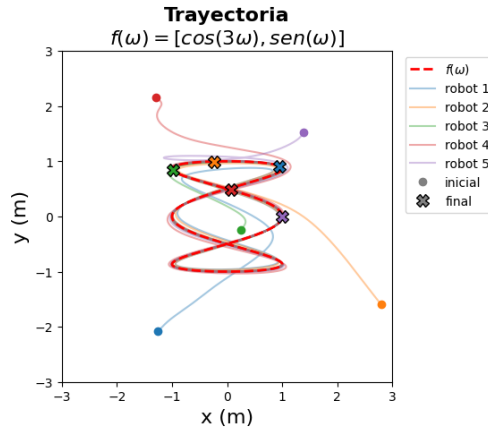
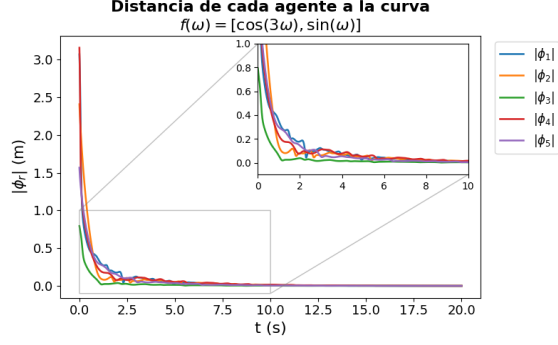
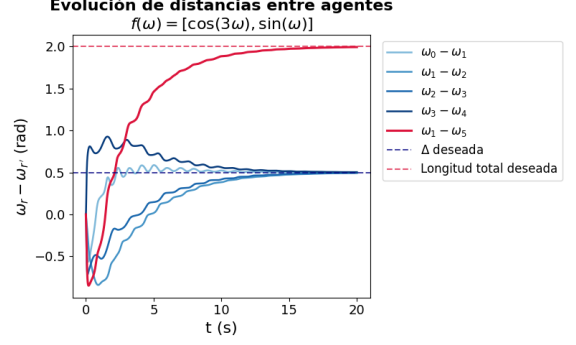


Figura 7: Trayectorias de 5 robots durante $t = 20s$, con condiciones iniciales aleatorias, implementando control de distancias. Para curva $\mathbf{f}(\omega) = [\cos(3\omega), \sin(\omega)]$ y distancias $\Delta_{r,r'} = 0,5$ rad. Las ganancias son $k_x = 2s^{-1}$, $k_y = 2s^{-1}$, $k_c = 1s^{-1}$, con $\lambda = 1rad s^{-1}$

Para comprender mejor cómo se desarrolla la dinámica, debe atenderse a la figura 8, donde se representa la evolución temporal de la distancia a la curva de cada agente, $|\phi_r| = \sqrt{\phi_x^2 + \phi_y^2}$, que debe anularse para $t \rightarrow \infty$. También se grafica la evolución de las distancias entre agentes $\omega_r - \omega_{r'}$, junto con la longitud de la cadena que forman:



(a) Evolución temporal de las distancias a la curva. Implementa zoom al periodo transitorio



(b) Evolución temporal de la distancia entre agentes.

Figura 8: Evolución temporal de la simulación de $t = 20s$, para seguimiento de trayectoria coordinado. Simulado para la curva $f(\omega) = [\cos(3\omega), \sin(\omega)]$ y distancias $\Delta_{rr'} = 0,5$ rad. Las ganancias son $k_x = 2s^{-1}$, $k_y = 2s^{-1}$, $k_c = 1s^{-1}$, con $\lambda = 1\text{rad}s^{-1}$.

En las gráficas se observa como el sistema converge a los valores deseados, tanto de distancia a la curva, como manteniendo la distancia entre robots.

5. Análisis de convergencia

En el apartado anterior se visualiza la convergencia de las variables a los parámetros deseados. Sin embargo, es posible hacer un estudio más riguroso de esta convergencia. En esta sección, se muestra la prueba de que el campo $\vec{w}_r$ cumple los objetivos de control, dadas cualesquiera condiciones iniciales. Esto es que para $t \rightarrow \infty$, las trayectorias convergen a la curva, y las distancias entre robots convergen a los valores deseados $\Delta_{rr'}$.

5.1. Estudio de la dinámica del error

Para estudiar la convergencia, debe atenderse al error, que para el sistema de estudio se define como:

$$e = \begin{bmatrix} \Phi \\ D^T \tilde{\omega} \end{bmatrix} \in \mathbb{R}^{N_R(n+1)} \quad (16)$$

Donde:

$$\begin{aligned} \Phi &= [\Phi^1, \dots, \Phi^{N_R}]^T \in \mathbb{R}^{nN_R} \\ \Phi^r &= [\phi_1^r, \dots, \phi_n^r] \in \mathbb{R}^n \end{aligned} \quad (17)$$

El primer bloque Φ recoge los errores de seguimiento de la curva para cada robot. El segundo bloque $D^T \tilde{\omega}$ representa los errores de coordinación, es decir, la desviación de las distancias entre robots $\omega_r - \omega_{r'}$ respecto a las deseadas $\Delta_{rr'}$. Esto último resulta más evidente al desarrollar la expresión:

$$D^T \tilde{\omega} = \begin{bmatrix} \tilde{\omega}_2 - \tilde{\omega}_1 \\ \tilde{\omega}_3 - \tilde{\omega}_2 \\ \vdots \\ \tilde{\omega}_{N_R} - \tilde{\omega}_{N_R-1} \\ \tilde{\omega}_1 - \tilde{\omega}_{N_R} \end{bmatrix} = \begin{bmatrix} (\omega_2 - \omega_1) - \Delta_{12} \\ (\omega_3 - \omega_2) - \Delta_{23} \\ \vdots \\ (\omega_{N_R} - \omega_{N_R-1}) - \Delta_{N_R-1, N_R} \\ (\omega_1 - \omega_{N_R}) - \Delta_{N_R, 1} \end{bmatrix} \quad (18)$$

Para comprobar la convergencia, se estudia la dinámica del error:

$$\dot{e} = \begin{bmatrix} \dot{\Phi} \\ D^T \dot{\tilde{\omega}} \end{bmatrix} \quad (19)$$

A continuación estudiamos por separado la dinámica de Φ^r :

$$\dot{\Phi}^r = \frac{d}{dt} \begin{bmatrix} \phi_1^r \\ \vdots \\ \phi_n^r \end{bmatrix} = \begin{bmatrix} \nabla \phi_1^{rT} \dot{\xi}^r \\ \vdots \\ \nabla \phi_n^{rT} \dot{\xi}^r \end{bmatrix} = \begin{bmatrix} \nabla \phi_1^{rT} \vec{w}_r \\ \vdots \\ \nabla \phi_n^{rT} \vec{w}_r \end{bmatrix} = \begin{bmatrix} \nabla \phi_1^{rT} \vec{v}_r^{guia} + \nabla \phi_1^{rT} k_c \vec{v}_r^{coord} \\ \vdots \\ \nabla \phi_n^{rT} \vec{v}_r^{guia} + \nabla \phi_n^{rT} k_c \vec{v}_r^{coord} \end{bmatrix} \quad (20)$$

Para ello, se emplean las siguientes definiciones para un solo robot de índice r :

$$K^r := \text{diag}\{k_1^r, \dots, k_n^r\} \in \mathbb{R}^{n \times n} \quad (21)$$

$$\mathbf{f}'^r(\omega_r) := [f_1'^r(\omega_r), \dots, f_n'^r(\omega_r)]^T \in \mathbb{R}^n$$

Con estas definiciones se generalizan varios aspectos que no se han considerado en las simulaciones. Pueden asignarse ganancias distintas a robots distintos, al igual que pueden definirse curvas distintas para cada robot. Ahora, se desglosa cada sumando de la ecuación (20).

$$\begin{aligned} \nabla \phi_i^{rT} \vec{v}_r^{guia} &= \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ -f_i'^r(\omega_r) \end{bmatrix}^T \begin{bmatrix} \lambda f_1'^r(\omega_r) - k_1^r \phi_1 \\ \vdots \\ \lambda f_n'^r(\omega_r) - k_n^r \phi_n \\ \lambda + \sum_{j=1}^n k_j^r \phi_j f_j'^r(\omega_r) \end{bmatrix} = \\ &= -k_i^r \phi_i - \mathbf{f}'_i^r(\omega_r) (\mathbf{f}'^r(\omega_r)^T K^r \Phi^r) \end{aligned} \quad (22)$$

$$\nabla \phi_i^{rT} \vec{v}_r^{coord} = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ -f_i'^r(\omega_r) \end{bmatrix}^T \begin{bmatrix} 0 \\ \vdots \\ 0 \\ \vdots \\ 0 \\ c_r(\omega) \end{bmatrix} = -f_i'^r(\omega_r) c_r(\omega) \quad (23)$$

Empleando las siguientes definiciones:

$$\begin{aligned} F &:= \text{diag}\{\mathbf{f}'^1, \dots, \mathbf{f}'^{N_R}\} \in \mathbb{R}^{n_{N_R} \times n_{N_R}} \\ K &:= \text{diag}\{K^1, \dots, K^{N_R}\} \in \mathbb{R}^{n_{N_R} \times n_{N_R}} \\ \mathbf{c}(\boldsymbol{\omega}) &:= [c_1(\boldsymbol{\omega}), \dots, c_{N_R}(\boldsymbol{\omega})]^T \in \mathbb{R}^{N_R} \end{aligned} \quad (24)$$

Al introducir las expresiones (22) y (23) en (20), y tomando $\mathbf{c}(\boldsymbol{\omega}) = -L\tilde{\boldsymbol{\omega}}$ queda:

$$\dot{\boldsymbol{\Phi}} = \begin{bmatrix} -K^1\boldsymbol{\Phi}^1 - \mathbf{f}'^1\mathbf{f}'^{1T}K^1\boldsymbol{\Phi}^1 - k_c c_1(\boldsymbol{\omega}) \mathbf{f}'^1 \\ \vdots \\ -K^{N_R}\boldsymbol{\Phi}^{N_R} - \mathbf{f}'^{N_R}\mathbf{f}'^{N_R T}K^{N_R}\boldsymbol{\Phi}^{N_R} - k_c c_{N_R}(\boldsymbol{\omega}) \mathbf{f}'^{N_R} \end{bmatrix} = -K\boldsymbol{\Phi} - FF^TK\boldsymbol{\Phi} + k_c FL\tilde{\boldsymbol{\omega}} \quad (25)$$

A la hora de calcular $D^T\dot{\tilde{\boldsymbol{\omega}}}$, la expresión puede simplificarse, dado que $\dot{\tilde{\boldsymbol{\omega}}} = \dot{\boldsymbol{\omega}}$:

$$D^T\dot{\tilde{\boldsymbol{\omega}}} = D^T\dot{\boldsymbol{\omega}} \quad (26)$$

Se calculan las derivadas de las componentes ω_r :

$$\begin{aligned} \dot{\omega}_r &= \nabla \omega_r^T \dot{\boldsymbol{\xi}}_r = \nabla \omega_r^T \vec{w}_r = \begin{bmatrix} 0 & \dots & 0 & 1 \end{bmatrix} \begin{bmatrix} \lambda f_1'^r(\omega_r) - k_1^r \phi_1^r \\ \vdots \\ \lambda f_n'^r(\omega_r) - k_n^r \phi_n^r \\ \lambda + \sum_{j=1}^n k_j^r \phi_j^r f_j'^r(\omega_r) + k_c c_r(\boldsymbol{\omega}) \end{bmatrix} = \\ &= \lambda + \mathbf{f}'^r(\omega_r)^T K^r \boldsymbol{\Phi}^r + k_c c_r(\boldsymbol{\omega}) \end{aligned} \quad (27)$$

Y finalmente apilando las componentes $\dot{\omega}_r$ para obtener $\dot{\boldsymbol{\omega}}$:

$$\dot{\boldsymbol{\omega}} = \begin{bmatrix} \lambda + \mathbf{f}'^1(\omega_1)^T K^1 \boldsymbol{\Phi}^1 + k_c c_1(\boldsymbol{\omega}) \\ \vdots \\ \lambda + \mathbf{f}'^{N_R}(\omega_{N_R})^T K^{N_R} \boldsymbol{\Phi}^{N_R} + k_c c_{N_R}(\boldsymbol{\omega}) \end{bmatrix} = \lambda \mathbf{1} + F^T K \boldsymbol{\Phi} - k_c L \tilde{\boldsymbol{\omega}} \quad (28)$$

Donde $\mathbf{1} \in \mathbb{R}^{N_R}$ es un vector cuyos elementos son todos 1. El último paso sería multiplicar esta última expresión por D^T . Tomando las expresiones obtenidas en (25) y (28), la dinámica completa del error queda:

$$\dot{\mathbf{e}} = \begin{bmatrix} \dot{\boldsymbol{\Phi}} \\ D^T \dot{\boldsymbol{\omega}} \end{bmatrix} = \begin{bmatrix} -K\boldsymbol{\Phi} - FF^TK\boldsymbol{\Phi} + k_c FL\tilde{\boldsymbol{\omega}} \\ D^T F^T K \boldsymbol{\Phi} - k_c D^T L \tilde{\boldsymbol{\omega}} \end{bmatrix} \quad (29)$$

5.2. Estabilidad de Lyapunov

A continuación se presenta la prueba de convergencia extraída de [9]. A través del método de estabilidad de Lyapunov, puede demostrarse que la dinámica del error $\mathbf{e}$ converge a los valores deseados. Para ello se debe escoger una función candidata de Lyapunov $V : \mathbb{R}^{N_R(n+1)} \rightarrow \mathbb{R}$ tal que:

$$V(\Phi, D^T \tilde{\omega}) = \begin{cases} 0 & \text{si } \Phi = \mathbf{0} \text{ y } D^T \tilde{\omega} = \mathbf{0} \\ > 0 & \text{para todo } (\Phi, D^T \tilde{\omega}) \neq (\mathbf{0}, \mathbf{0}) \end{cases} \quad (30)$$

Gracias a sus características, una función candidata de Lyapunov bien escogida permite calcular analíticamente si la dinámica converge al estado deseado. Se calcula su derivada temporal, que por la regla de cadena resulta: $\dot{V}(\mathbf{x}) = \nabla V \cdot \dot{\mathbf{x}}$. Si se demuestra que $\nabla V \cdot \dot{\mathbf{x}}$ es negativo, significa que $\mathbf{x}$ evoluciona hacia curvas de nivel de V cada vez menores, es decir, hacia el mínimo. Como el mínimo se corresponde con el estado deseado, la dinámica convergerá hacia el deseado si $\dot{V} < 0$. En resumen, el método de Lyapunov trata de escoger una función que cumpla las condiciones de (30), y demostrar que su derivada temporal es negativa. En el caso de que se cumpla $\dot{V} \leq 0$, debe recurrirse al principio de invariancia de LaSalle [11].

Definimos una función de Lyapunov que cumpla todas las condiciones mencionadas en (30):

$$V(\Phi, D^T \tilde{\omega}) = \frac{1}{2} (\Phi^T K \Phi + k_c \tilde{\omega}^T L \tilde{\omega}) \quad (31)$$

Y se calcula su evolución temporal:

$$\begin{aligned} \dot{V}(\Phi, D^T \tilde{\omega}) &= (\dot{\Phi}^T K \Phi + k_c \dot{\tilde{\omega}}^T L \tilde{\omega}) = (\dot{\Phi}^T K \Phi + k_c \dot{\tilde{\omega}}^T L \tilde{\omega}) = \\ &= \begin{bmatrix} -K\Phi - FF^T K\Phi + k_c FL\tilde{\omega} \\ D^T F^T K\Phi - k_c D^T L\tilde{\omega} \end{bmatrix}^T \begin{bmatrix} K & \\ & k_c \mathbb{1}_{|N_R|} \end{bmatrix} \begin{bmatrix} \Phi \\ D^T \tilde{\omega} \end{bmatrix} \end{aligned} \quad (32)$$

Donde $\mathbb{1}_{N_R}$ es la matriz identidad de $N_R \times N_R$ dimensiones. Se realiza el producto matricial, agrupando los términos cuadráticos e intentando eliminar los términos que no sean negativos, ya que el objetivo es demostrar que $\dot{V}$ sea negativa. Desarrollando la expresión anterior:

$$\begin{aligned} \dot{V}(\Phi, D^T \tilde{\omega}) &= (-K\Phi - FF^T K\Phi + k_c FL\tilde{\omega})^T K\Phi + (D^T F^T K\Phi - k_c D^T L\tilde{\omega})^T k_c D^T \tilde{\omega} \\ &= -\|K\Phi\|^2 - \|F^T K\Phi\|^2 + k_c (FL\tilde{\omega})^T K\Phi + k_c (F^T K\Phi)^T L\tilde{\omega} - k_c^2 \|L\tilde{\omega}\|^2 \\ &= -\|K\Phi\|^2 - \|F^T K\Phi\|^2 + 2k_c (F^T K\Phi)^T (L\tilde{\omega}) - k_c^2 \|L\tilde{\omega}\|^2 \\ &= -\|K\Phi\|^2 - \|F^T K\Phi - k_c L\tilde{\omega}\|^2 \\ &\leq -\|K\Phi\|^2 \leq 0 \end{aligned} \quad (33)$$

Vemos que la derivada de la función Lyapunov no queda negativa, ya que puede ser también nula si $\Phi = \mathbf{0}$. Sin embargo, aún puede demostrarse la convergencia a los valores deseados con el principio de invariancia de LaSalle. En primer lugar, se definen los puntos donde $\dot{V} = 0$ como:

$$\mathcal{B} = \{(\Phi, D^T \tilde{\omega}) : \dot{V} = 0\} = \{(\Phi, D^T \tilde{\omega}) : \Phi = \mathbf{0}\} \quad (34)$$

También debe definirse el mayor conjunto invariante de $\mathcal{B}$, que son los puntos de $\mathcal{B}$ donde, si el estado llega, no puede salir. Esto se traduce a que $\dot{\Phi} = 0, \forall \Phi = \mathbf{0}$. A este conjunto lo denominamos $\mathcal{A}$:

$$\mathcal{A} = \{(\Phi, D^T \tilde{\omega}) \in B : \dot{\Phi} = \mathbf{0}\} \quad (35)$$

El principio de invariancia de LaSalle afirma que las trayectorias convergen al mayor conjunto invariante en B , es decir, $\mathcal{A}$ [11]. Ahora quedaría demostrar que:

$$\mathcal{A} = \{\Phi = 0, D^T \tilde{\omega} = 0\} \quad (36)$$

En [9] se demuestra el procedimiento algebraico final, donde se concluye que los estados finales deseados componen $\mathcal{A}$, por lo que el sistema converge a los mismos.

6. Esquiva entre agentes

6.1. Método de barreras de potencial

En este apartado se define el último elemento para consolidar el algoritmo completo para el control del sistema. Gracias al guiado coordinado, los robots mantienen las distancias entre ellos sobre la curva, pero eso no garantiza que no se den colisiones entre ellos, ya sea durante el periodo transitorio de convergencia, o porque la curva tenga intersecciones donde puedan cruzarse.

Se recurre al método de las barreras de control, propuesto en [12]. La construcción del algoritmo comienza por definir la región prohibida. Esta estará centrada en cada agente, de forma que tendremos N_R regiones prohibidas en el espacio de trayectorias. Para estas simulaciones, la región prohibida tendrá una forma circular de radio $R = 0,1\text{m}$. En el caso tridimensional, la región prohibida tendría forma esférica. También se define una función de barrera candidata B , que se encargará de limitar las velocidades a las que los agentes pueden acercarse los unos a los otros. La función barrera $B : \mathbb{R}^{2(n+1)} \rightarrow \mathbb{R}$ toma la forma:

$$B_{rr'}(\xi_r, \xi_{r'}) = \frac{1}{\eta_{rr'}(\xi_r, \xi_{r'})} \quad (37)$$

Donde la función segura $\eta_{rr'} : \mathbb{R}^{2(n+1)} \rightarrow \mathbb{R}$ se encarga de construir la función barrera. Esta se define como:

$$\eta_{rr'}(\xi_r, \xi_{r'}) = |P(\xi_r - \xi_{r'})|^2 - R^2 \quad (38)$$

La matriz $P \in \mathbb{R}^{(n+1) \times (n+1)}$ es el proyector sobre el subespacio de coordenadas reales. El subíndice r hace referencia al robot que se estudia, mientras que r' representa al resto de robots. Cada uno de estos últimos impondrá una restricción a la velocidad del robot r . El conjunto de robots distinto de r se denomina $\mathcal{D}_r$. Se impone una condición de seguridad: $\dot{B}_{rr'} \leq \frac{\alpha}{B_{rr'}} = \alpha \eta_{rr'}$, que pone una cota máxima a las velocidades con las que los agentes pueden acercarse los unos a los otros. El parámetro α endurece o relaja la condición de seguridad. Para α pequeños, la condición se endurece, limitando las velocidades permitidas entre agentes, y al contrario para α grandes. Desarrollando la derivada de la función barrera se obtiene:

$$\dot{B}_{rr'} = -\frac{\dot{\eta}_{rr'}}{\eta_{rr'}^2} = -\frac{2(\xi^r - \xi^{r'})^T P(\vec{w}^r - \vec{w}^{r'})}{\eta_{rr'}^2} \quad (39)$$

Introduciéndola en la condición de seguridad y operando, se llega a:

$$(\xi^{r'} - \xi^r)^T P \vec{w}^r + (\xi^r - \xi^{r'})^T P \vec{w}^{r'} \leq \frac{\alpha \eta_{rr'}^3}{2} \quad (40)$$

Si se asume que ambos sumandos contribuyen igual a la expresión anterior, la condición puede reescribirse como:

$$(\xi^{r'} - \xi^r)^T P \vec{w}^r \leq \frac{\alpha \eta_{rr'}^3}{4} \quad (41)$$

Para cada agente r , habrá $|\mathcal{D}_r|$ restricciones, que pueden reescribirse de forma matricial:

$$A \vec{w}^r \leq b$$

$$A_{r'} = (\xi^{r'} - \xi^r)^T P \quad b_{r'} = \frac{\alpha \eta_{rr'}^3}{4} \quad (42)$$

Donde $A_{r'}$ son las filas de la matriz $A \in \mathbb{R}^{|\mathcal{D}_r| \times (n+1)}$, $b_{r'}$ son los elementos del vector $\mathbf{b} \in \mathbb{R}^{|\mathcal{D}_r|}$, con tantas filas en A , y elementos en $\mathbf{b}$, como restricciones sobre el agente r .

Con esta construcción, la condición de seguridad depende únicamente del campo $\vec{w}^r$ del agente r , y de la distancia tanto al agente r' , como a la frontera de su región prohibida. Una vez definidas las restricciones, si $\vec{w}^r$ no viola ninguna de ellas, no se modifica el campo. Sin embargo, si no cumple una o varias restricciones, debe buscarse el vector modificado $\bar{\vec{w}}^r$ más cercano a $\vec{w}^r$ que si las cumpla. Por ello, debe resolverse el siguiente problema de optimización:

$$\min_{\bar{\vec{w}}} \frac{1}{2} \sum_{r=1}^{N_R} |\bar{\vec{w}}^r - \vec{w}^r|^2, \quad \text{rest: } A \bar{\vec{w}}^r \leq \mathbf{b} \quad \forall r' \in \mathcal{D}_r \quad (43)$$

Las modificaciones necesarias del campo se realizan en cada paso del integrador, y en cada paso, se van resolviendo uno por uno el problema asociado a cada agente del sistema.

6.2. Multiplicadores de Lagrange

En esta sección λ representa los multiplicadores de Lagrange, no la velocidad sobre la curva definida en (6). El algoritmo se reduce a resolver el problema de optimización planteado en (43). Para simplificarlo, se desarrolla la norma del sumatorio:

$$|\bar{\vec{w}}^r - \vec{w}^r|^2 = (\bar{\vec{w}}^r)^T \bar{\vec{w}}^r - 2(\vec{w}^r)^T \bar{\vec{w}}^r + |\vec{w}^r|^2$$

El término $|\vec{w}^r|^2$ no depende de la variable de optimización $\bar{\vec{w}}^r$, por lo que es constante y no altera la posición del mínimo. Puede eliminarse, tomando únicamente el resto de sumandos. Para el robot r , introducimos el Lagrangiano del problema, donde el campo puede separarse en sus componentes espaciales x e y :

$$\mathcal{L}\left(\begin{bmatrix} \bar{w}_x^r \\ \bar{w}_y^r \end{bmatrix}, \lambda\right) = \frac{1}{2} \begin{bmatrix} \bar{w}_x^r & \bar{w}_y^r \end{bmatrix} \begin{bmatrix} \bar{w}_x^r \\ \bar{w}_y^r \end{bmatrix} - \begin{bmatrix} w_x^r & w_y^r \end{bmatrix} \begin{bmatrix} \bar{w}_x^r \\ \bar{w}_y^r \end{bmatrix} + \lambda^T \left(A \begin{bmatrix} \bar{w}_x^r \\ \bar{w}_y^r \end{bmatrix} - \mathbf{b} \right) \quad (44)$$

Donde $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_{|\mathcal{D}_r|})^T \in \mathbb{R}^{|\mathcal{D}_r|}$ contiene un multiplicador por cada restricción. Buscamos los puntos que cumplan:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \bar{w}_x^r} &= \bar{w}_x^r - w_x^r + (A^T \boldsymbol{\lambda})_x = 0 \\ \frac{\partial \mathcal{L}}{\partial \bar{w}_y^r} &= \bar{w}_y^r - w_y^r + (A^T \boldsymbol{\lambda})_y = 0\end{aligned}\tag{45}$$

Estas dos primeras expresiones dan cuenta de la propiedad de mínimo de los multiplicadores de Lagrange, $\nabla f = -\lambda \nabla g$, donde f es la función a optimizar, y g la restricción. Se pueden expresar de forma compacta como:

$$\begin{bmatrix} \bar{w}_x^r \\ \bar{w}_y^r \end{bmatrix} = \begin{bmatrix} w_x^r \\ w_y^r \end{bmatrix} - A^T \boldsymbol{\lambda}\tag{46}$$

6.3. Algoritmo de resolución

Una vez planteados los multiplicadores de Lagrange, pueden enunciarse los pasos que sigue el algoritmo de optimización. En primer lugar, se comprueba si el campo sin modificar $\vec{w}^r$ ya cumple todas las restricciones $A \vec{w}^r \leq \mathbf{b}$. En caso de que las cumpla, todos los multiplicadores son nulos $\boldsymbol{\lambda} = \mathbf{0}$, y el óptimo es el campo inicial sin modificar:

$$\vec{\bar{w}}^r = \vec{w}^r\tag{47}$$

Cada restricción divide el espacio de velocidades en una zona permitida y otra no permitida, trazando una recta sobre el plano de vectores $\vec{w}_r$ posibles. Si una o varias restricciones se violan, el óptimo se encontrará sobre el límite de una restricción, o sobre la intersección de dos restricciones. En el caso de una trayectoria tridimensional de los agentes, las restricciones dividen el volumen de vectores $\vec{w}_r$ posibles con un plano, y los óptimos podrán encontrarse en estos planos o en sus intersecciones, que pueden ser rectas o puntos.

Se denota con el subíndice A al conjunto de restricciones activas, es decir, la restricción o restricciones sobre las que descansará la solución óptima. Más adelante se detalla cómo se obtiene este conjunto activo. Se seleccionan las filas de A y los elementos de $\mathbf{b}$ que se corresponden con las restricciones activas, formando A_A y $\mathbf{b}_A$. Tomando estas condiciones, obtenemos el siguiente sistema de ecuaciones:

$$\vec{\bar{w}}^r = \vec{w}^r - A_A^T \boldsymbol{\lambda}_A\tag{48}$$

$$A_A \vec{\bar{w}}^r = \mathbf{b}_A\tag{49}$$

La primera ecuación proviene de sustituir las restricciones activas en (46). La segunda, de la definición de punto óptimo, dado que para este se satura la desigualdad de la restricción. Sustituyendo la primera en la segunda, se despeja $\boldsymbol{\lambda}_A$:

$$A_A (\vec{w}^r - A_A^T \boldsymbol{\lambda}_A) = \mathbf{b}_A \implies \boldsymbol{\lambda}_A = (A_A A_A^T)^{-1} (A_A \vec{w}^r - \mathbf{b}_A)\tag{50}$$

Con este valor de λ_A se obtiene el campo modificado sustituyendo de nuevo en la ecuación (48).

6.4. Restricciones activas

Una parte muy relevante del algoritmo es encontrar las restricciones activas, que son aquellas que se cumplen con igualdad en el óptimo ($A_{r'} \vec{w}^r = b_{r'}$), lo cuál no es trivial. Para ello se emplea el método de conjunto activo [13, Cap. 16]. Este es un método de carácter iterativo, que va probando distintos conjuntos de restricciones candidatas, empezando por la que más se viola: $\max[A_{r'} \vec{w}^r - b_{r'}]$. Se obtienen los multiplicadores asociados a esos conjuntos de prueba, descartándolos si sus multiplicadores quedan negativos, o si se incumplen restricciones en el óptimo propuesto durante la iteración. En la implementación, este problema cuadrático se resuelve con la función `solve_qp` de la biblioteca `qpsoolvers`, empleando el optimizador DAQP [14].

6.5. Simulaciones para el campo $\vec{w}$ implementando evitado de colisiones

Los primeros ejemplos son simples para poder observar con detalle el efecto de las barreras. En ellos se fuerza la colisión de dos agentes que convergen a una trayectoria en la que deben cruzarse y modificar sus campos en consecuencia. La curva a seguir es $f(\omega) = [\cos(\omega), \sin(2\omega)]$, con distancia objetivo entre robots de $\Delta = \pi$ (rad). A partir de aquí todas las simulaciones se harán con las mismas ganancias y velocidad sobre la curva, $k_x = k_y = 2 \text{ s}^{-1}$, $k_c = 1 \text{ s}^{-1}$ y $\lambda = 1 \text{ rad s}^{-1}$. A continuación se muestra el efecto sobre la trayectoria para distintos valores del parámetro α :

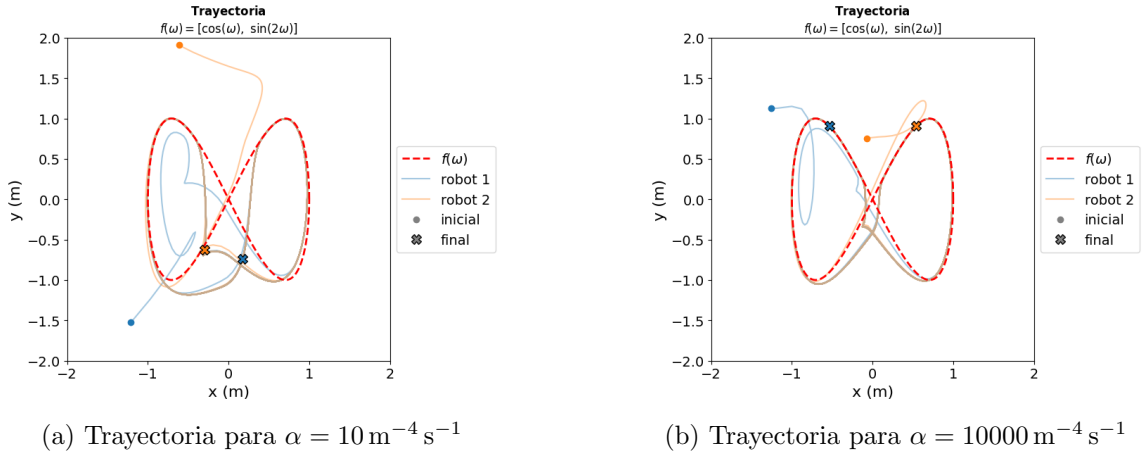


Figura 9: Comparativa de trayectorias para dos robots que convergen a la curva $f(\omega) = [\cos(\omega), \sin(2\omega)]$, con distancia $\Delta = \pi$ rad, para valores distintos de α . Condiciones iniciales aleatorias.

Se aprecia que para $\alpha = 10 \text{ m}^{-4} \text{ s}^{-1}$, los robots se desvían mucho más para evitar la colisión, en comparación con $\alpha = 10000 \text{ m}^{-4} \text{ s}^{-1}$. Sin embargo, logran converger de nuevo a la curva tras esquivarse en ambos casos. Además, se desvían siempre en el mismo lugar, lo que sugiere que las trayectorias son periódicas; y en caso de ser así, no cabría esperar que los robots cambien sus trayectorias modificadas a lo largo del tiempo. Podría definirse otro tipo de objetivo de control más laxo para los casos con barreras, dado que los objetivos originales de convergencia no se cumplen. Este podría ser que las trayectorias se mantengan a una distancia mínima de la curva objetivo y con trayectorias periódicas.

Para estudiar en más detalle el comportamiento de las trayectorias frente al parámetro α , se han realizado simulaciones para distintos valores de α , manteniendo la curva, el número de robots, y la

distancia deseada entre ellos. Se ha medido la desviación máxima respecto de los valores deseados para obtener la amplitud de oscilaciones, una vez el sistema se estabiliza a una trayectoria periódica. A continuación se muestran las amplitudes para las distintas simulaciones:

$\alpha \text{ (m}^{-4}\text{s}^{-1}\text{)}$	1	10	100	1000	10000	100000
Amp. $\omega_1 - \omega_2$ (rad)	0.2299	0.1766	0.1419	0.1187	0.1033	0.0926
Amplitud $\ \Phi\ _r$ (m)	0.8196	0.6519	0.5237	0.4375	0.3887	0.3699

Tabla 1: Amplitud de las oscilaciones de la distancia $\omega_1 - \omega_2$ y $\|\Phi\|$ en régimen estable, para distintos valores de α . Se simulan 2 agentes, con curva $f(\omega) = [\cos(\omega), \sin(2\omega)]$ y $\Delta = \pi$ rad

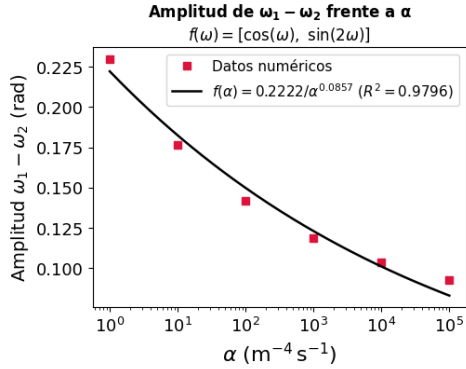


Figura 10: Amplitudes en la desviación de $\omega_1 - \omega_2$, frente a α en escala logarítmica en base 10.

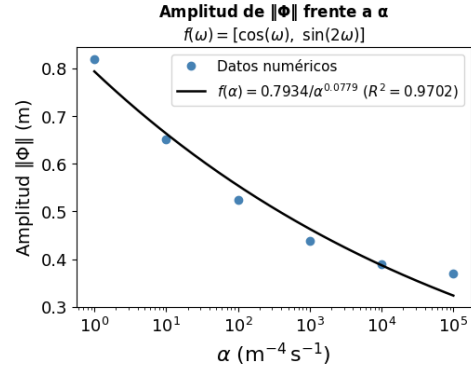


Figura 11: Amplitudes en la desviación de $\|\Phi\|_r$, frente a α en escala logarítmica en base 10.

Al observar los datos de la tabla 1, podía intuirse un aparente decaimiento asintótico. Por ello, se probó a ajustar los datos a una ley de potencias del tipo $f(\alpha) = K/\alpha^a$. En la sección de conclusiones se comentará la bondad del ajuste. Vemos que parece cumplirse el comportamiento esperado; al aumentar α , las desviaciones respecto de los valores deseados son menores, dado que los robots varían menos su campo para esquivarse. A continuación, se simula una curva con más intersecciones:

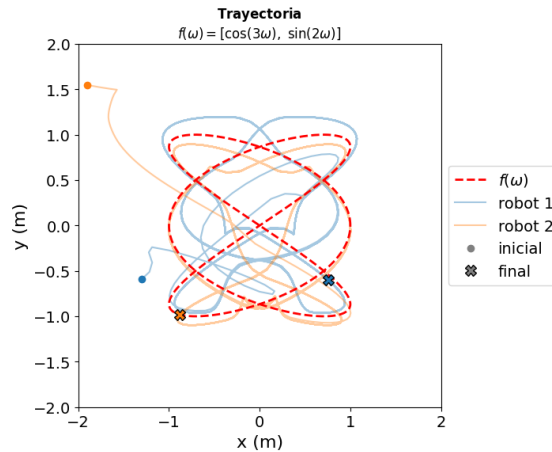


Figura 12: Trayectoria de dos robots, para la curva $f(\omega) = [\cos(3\omega), \sin(2\omega)]$, con parámetro $\alpha = 10 \text{ m}^{-4}\text{s}^{-1}$. Condiciones iniciales aleatorias

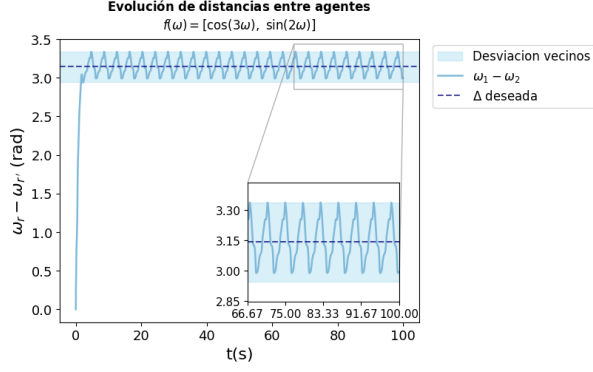


Figura 13: Evolución de $\omega_1 - \omega_2$, para la curva $\mathbf{f}(\omega) = [\cos(3\omega), \sin(2\omega)]$, con parámetro $\alpha = 10 \text{ m}^{-4} \text{ s}^{-1}$. Incluye subfigura con zoom sobre las oscilaciones

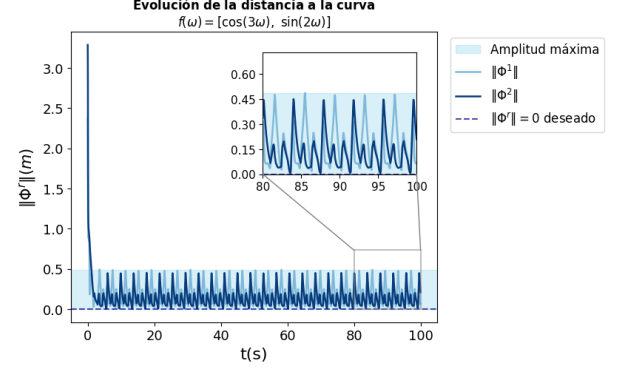


Figura 14: Evolución de $|\Phi|_r$, para la curva $\mathbf{f}(\omega) = [\cos(3\omega), \sin(2\omega)]$, con parámetro $\alpha = 10 \text{ m}^{-4} \text{ s}^{-1}$. Incluye subfigura con zoom sobre las oscilaciones

Vemos que las desviaciones vuelven a tener apariencia periódica en las figuras 12, 13 y 14. Resulta interesante que, pese a no cumplirse los objetivos iniciales de control, el sistema parece estabilizarse en una nueva trayectoria provocada por los campo modificados, al igual que para la trayectoria dibujada en la figura 9. Para el último ejemplo, se simula la misma curva que para el caso anterior, pero aumentando a 10 el número de robots. La distancia entre ellos es de $\Delta = \pi/10$.

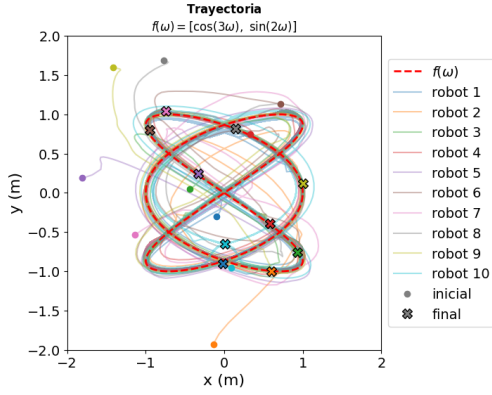


Figura 15: Trayectoria de diez robots, para la curva $\mathbf{f}(\omega) = [\cos(3\omega), \sin(2\omega)]$, con parámetro $\alpha = 1000 \text{ m}^{-4} \text{ s}^{-1}$. Condiciones iniciales aleatorias

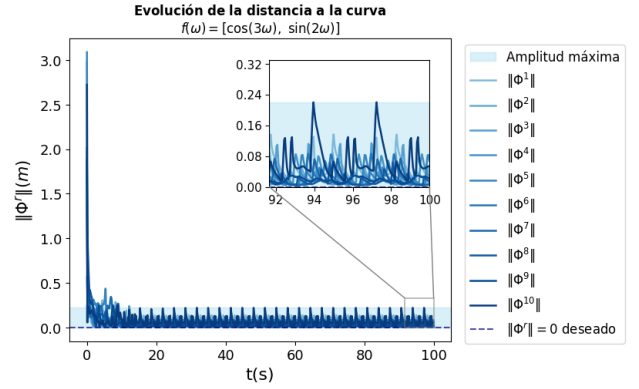


Figura 16: Evolución de $|\Phi|_r$, para la curva $\mathbf{f}(\omega) = [\cos(3\omega), \sin(2\omega)]$, con $\alpha = 1000 \text{ m}^{-4} \text{ s}^{-1}$. Incluye subfigura con zoom sobre las oscilaciones

Al observar la figura 15, no puede afirmarse que haya una trayectoria periódica definida, algo que era más evidente para los casos con dos robots. Si atendemos a la figura 16 de la distancia a la curva, la periodicidad resulta más evidente. La figura 17 es donde más se evidencia la periodicidad:

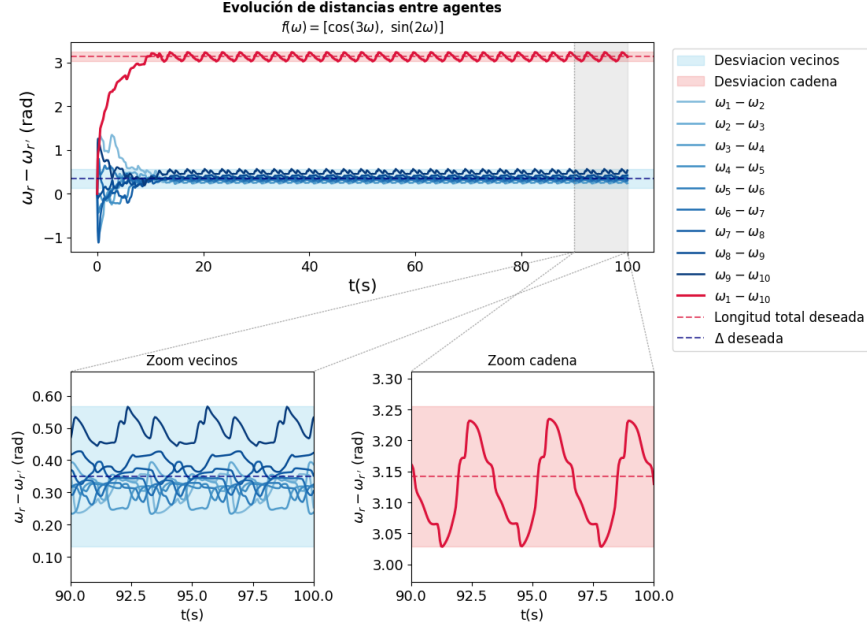


Figura 17: Evolución de $\omega_1 - \omega_2$, y de la longitud total de la cadena, para la curva $\mathbf{f}(\omega) = [\cos(3\omega), \sin(2\omega)]$, con parámetro $\alpha = 1000 \text{ m}^{-4} \text{ s}^{-1}$. Incluye subfiguras con zoom sobre las oscilaciones para la distancia entre vecinos y la longitud de la cadena

7. Conclusiones

Tras analizar los datos obtenidos para el campo de guiado coordinado, con evitado de agentes vecinos, se obtienen varias conclusiones.

La primera y más evidente es que, el campo de guiado que implementa todos los elementos de control logra con éxito la convergencia a cualquier curva parametrizada de tipo C^2 , con derivadas acotadas [9], y logra la disposición equidistante de los agentes sobre la misma. Esto lo hemos comprobado analíticamente con el método de estabilidad de Lyapunov, y representado gráficamente mediante las simulaciones.

Sin embargo, al introducir el evitado entre agentes, se incumplen los objetivos de control de convergencia a la curva y a las distancias deseadas. Esto se debe a que los campos modificados empujan a los robots a escapar de la curva o a alterar su posición respecto a sus vecinos.

También se ha tratado de obtener la relación entre las desviaciones respecto de los parámetros deseados, y el parámetro α . Se comprueba computacionalmente que las desviaciones son mayores a mayor α , lo que coincide con lo esperable. Al tratar de obtener una relación con una ley de potencias, se obtienen ajustes con un R^2 mayor de 0,97, lo que resulta aceptable. Sin embargo, la bondad del ajuste no implica que esta ley sea correcta. Los grados de libertad representan un tercio de los datos ajustados, y no se tienen incertidumbres de las medidas para poder hablar en términos de compatibilidad. Una posible mejora es hacer un estudio estadísticamente más riguroso, implementando métodos de Montecarlo, y tomando un registro mayor de valores para α .

Por último, pese a que no se cumplan los objetivos de control, la implementación de barreras parece no impedir una cierta estabilidad en las trayectorias. Se observa que para los casos simulados, tras un periodo transitorio, estas parecen volverse periódicas, manteniendo su amplitud constante. Pero con lo estudiado en este trabajo, no se puede afirmar que cualquier curva, con los parámetros

definidos cualesquiera, vaya a dar trayectorias periódicas, por varios motivos. En primer lugar, no se ha hecho un estudio riguroso de la periodicidad, más allá de observar las gráficas. Podría tratar de emplearse algún algoritmo que estime si una función es o no periódica. Por otro lado, el número de casos estudiados es muy limitado. Para poder afirmar estas propiedades periódicas se necesitaría un estudio más exhaustivo de estas propiedades, con un número mayor de simulaciones, con diferentes parámetros o curvas.

Referencias

- [1] W. Yao, *Guiding Vector Fields for Robot Motion Control*, ser. Springer Tracts in Advanced Robotics. Cham, Switzerland: Springer, 2023, vol. 154.
- [2] A. P. Aguiar, J. P. Hespanha, and P. V. Kokotović, “Performance limitations in reference tracking and path following for nonlinear systems,” *Automatica*, vol. 44, no. 3, pp. 598–610, 2008.
- [3] P. B. Sujit, S. Saripalli, and J. B. Sousa, “Unmanned aerial vehicle path following: A survey and analysis of algorithms for fixed-wing unmanned aerial vehicles,” *IEEE Control Systems Magazine*, vol. 34, no. 1, pp. 42–59, 2014.
- [4] W. Yao, H. Garcia de Marina, B. Lin, and M. Cao, “Singularity-free guiding vector field for robot navigation,” *IEEE Transactions on Robotics*, vol. 37, no. 4, pp. 1206–1221, 2021.
- [5] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [6] W. Yao, B. Lin, B. D. O. Anderson, and M. Cao, “Guiding vector fields for following occluded paths,” *IEEE Transactions on Automatic Control*, vol. 67, no. 8, pp. 4091–4106, 2022.
- [7] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *2019 18th European Control Conference (ECC)*. IEEE, 2019, pp. 3420–3431.
- [8] W. Yao, H. Garcia de Marina, Z. Sun, and M. Cao, “Distributed coordinated path following using guiding vector fields,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. Xi’an, China: IEEE, 2021, pp. 10 030–10 037.
- [9] W. Yao, “Guiding vector fields for robot motion control,” Ph.D. dissertation, University of Groningen, Groningen, The Netherlands, 2021.
- [10] L. Balfagón Zapatero, “TFG-Multiagente-GVF: repositorio de código del tfg,” <https://github.com/LucasBalfagon/TFG-Multiagente-GVF>, 2026.
- [11] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.
- [12] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [13] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., ser. Springer Series in Operations Research and Financial Engineering. New York, NY, USA: Springer, 2006.
- [14] D. Arnström, A. Bemporad, and D. Axehill, “A dual active-set solver for embedded quadratic programming using recursive LDL^t updates,” *IEEE Transactions on Automatic Control*, vol. 67, no. 8, pp. 4362–4369, 2022.